DAW-0613-PR-MIDTERM / M07-UF4

Nombre del alumno

Practica entregable (Enunciado y entregable)

Guía para el alumno

El alumno debe de entregar la práctica enunciada en este documento antes del cierre programado en el calendario.

Los entregables son:

- Carpeta de la práctica

  DAW-0613-PR-MidTerm/M07-UF4-"username"
  "username" = nombre de usuario del alumno en la plataforma
  Ejemplo: DAW-0613-PR-MidTerm/M07-UF4-garciafloresraul

  - La carpeta de la práctica contendrá los ficheros necesarios de la práctica, **no se debe subir la carpeta "node_modules".**

- Video demostración del correcto funcionamiento de la práctica.
  - Se puede entregar un enlace al video, ya que seguramente superará el límite permitido de tamaño para subirlo a la plataforma.
  - Si se entrega un enlace, es **obligatorio dar permisos de lectura.**

- Repositorio Github usando la herramienta Github Classroom con el enlace de entrega proporcionado.

Consultar Sesión Inicial para ver el video de cómo realizar estas operaciones.

Práctica Node.js — “CryptoExchange API”

Objetivo

Implementar una API tipo *crypto exchange* donde un usuario autenticado pueda:

- Gestionar un catálogo de **assets** (cryptos soportadas).
- Registrar y gestionar **transacciones** (BUY/SELL) asociadas a un asset.
- Consultar su **portfolio** agregado a partir de sus transacciones.
- Integrar precios actuales desde **CoinCap**.

⚠️ **La carpeta de frontend no hace falta tocarla**: solo se debe implementar parte backend.

⚠️ **No se debe implementar ningún tipo de blockchain** (solo simulación de exchange con BD + precios).

Modelo de datos (MongoDB)

1. Assets

Representa una crypto soportada por tu “exchange”.

Campos mínimos:

- _id (string, id de MongoDB)
- symbol (string, ej. "BTC")
- name (string, ej. "Bitcoin")
- coincapId (string, ej. "BTC")
- lastPriceUsd (number, opcional)
- lastPriceAt (date, opcional)

2. Transactions

Operaciones del usuario sobre un asset.

Campos mínimos:

- userId (ObjectId → users) ✅ relación obligatoria
- assetId (ObjectId → assets) ✅ relación obligatoria
- type ("BUY" | "SELL")
- quantity (number > 0)
- priceUsdAtExecution (number) ← se guarda al crear
- executedAt (date, default now)
- notes (string, opcional)

⚠️ **Importante:** el precio actual se consulta en CoinCap, pero el precio "de ejecución" queda **congelado** en la transacción.

Auth

Se reutiliza el sistema de auth de la demo:

- Rutas de exchange protegidas con JWT (access token).
- El userId del token se usa para filtrar datos (portfolio, transactions del usuario, etc.).

Api CoinCap (precios)

Usa la API de CoinCap para obtener el precio actual en USD de un asset.

Sigue la arquitectura vista en la teoría.

Requisitos

1) Assets

- GET /assets
  - Devuelve todos los assets.
- GET /assets/:id
  - Devuelve un asset por id.
- POST /assets
  - Crea un asset.
  - Validaciones mínimas: symbol, name, coincapId obligatorios. CoincapId único
- PATCH /assets/:id/refresh-last-price
  - Llama a CoinCap y actualiza:
    - lastPriceUsd
    - lastPriceAt
  - Devuelve el asset actualizado.

2) Transactions (CRUD completo)

- POST /transactions
- Requiere auth.
- Crea una transacción para el usuario logeado.
- Debe:
  1. Validar que el assetId existe.
  2. Obtener el **precio actual** de CoinCap (por asset.coincapId).

3. Guardar priceUsdAtExecution con ese precio.
4. Guardar userId desde el token (no desde el body).

- GET /transactions
  - Requiere auth.
  - Lista **solo** transacciones del usuario logueado.
  - Bonus interno (no opcional): paginación simple (page, limit) o skip/limit.
- GET /transactions/:id
  - Requiere auth.
  - Devuelve la transacción si pertenece al usuario.
  - Debe incluir el asset (usa populate/lookup).
- PATCH /transactions/:id
  - Requiere auth.
  - **No hay PUT.**
  - Solo se permite actualizar estos campos:
    - Notes
    - executedAt
  - No se puede cambiar: type, quantity, assetId, priceUsdAtExecution, userId.
- DELETE /transactions/:id
  - Requiere auth.
  - Borra solo si pertenece al usuario.

3) Relación entre colecciones

- transactions referencia a assets y a users.
- En al menos un endpoint (GET /transactions/:id o GET /portfolio) se debe devolver información del asset asociada (nombre/símbolo).

4) Portfolio

- GET /portfolio
  - Requiere auth.
  - Devuelve el estado del portfolio del usuario logeado, **agregando** sus transacciones.
  - Mínimo por asset:
    - asset (id + symbol + name)
    - netQuantity = BUY – SELL
    - currentPriceUsd (CoinCap)

- currentValueUsd = netQuantity × currentPriceUsd

Aquí se espera uso de agregaciones ($group) o lógica equivalente + resolución de la relación asset.

BONUS (escoge uno mínimo)

A) Refresh token en Auth

Modificar el auth para:

- POST /auth/login devuelva accessToken + refreshToken
- POST /auth/refresh permita obtener un nuevo accessToken usando el refreshToken

(Define claramente expiraciones y payload mínimo.)

B) Fallback si CoinCap falla al crear una transacción

En POST /transactions:

- Si CoinCap falla, usar asset.lastPriceUsd como precio.
- Guardar también priceSource: "COINCAP" | "CACHE" (campo extra).
- Si no existe lastPriceUsd, devolver error (ej. 503).

Entrega del proyecto

Es obligatorio realizar todas las entregas del proyecto para que la práctica se considere válida.

1. Repositorio Git

Entrega un enlace a un repositorio (GitHub o similar) con:

- Código fuente completo.
- Historial de commits razonable (no todo en un único commit).

2. README.md en la raíz

El README debe incluir:

2.1. Descripción del proyecto

- Breve descripción de qué hace la app.

2.2. Enlace al vídeo de demostración

Incluye un enlace (YouTube, Drive, Loom…) a un vídeo (5–10 min) donde se vea:

1. Ejecución de la API
2. Endpoints de assets
3. Endpoints de transactions
4. Endpoint de portfolio
5. Endpoint del bonus
6. Mostrar cómo se han hecho las relaciones de la BBDD.

2.4. Uso de IA

Apartado **"Uso de IA"** donde expliques:

- Qué herramientas de IA has usado.
- Para qué partes del proyecto (código, refactors, README, estilos…).
- Cómo has revisado y adaptado lo que ha generado la IA.

Uso de Issues (obligatorio)

El repositorio del proyecto incluye **issue templates** que representan los distintos bloques evaluables de la práctica (Assets, Transactions, Relaciones, Portfolio, Bonus, Uso de IA, Arquitectura, Uso de Git y Calidad de código).

El alumno debe:

- Crear los **issues** a partir de los templates proporcionados.
- Asignárselos a sí mismo.
- Implementar cada bloque correspondiente.
- **Completar la información solicitada dentro de cada issue** (especialmente en los issues de Arquitectura, Uso de Git y Calidad de código).
- Cerrar los issues únicamente cuando la funcionalidad esté correctamente implementada.

La evaluación se realizará en base a los **issues cerrados** y a la información incluida en ellos.
Un issue no creado, no completado o no cerrado se considerará **no entregado**.

Rúbrica de evaluación (10 puntos)

Importante:
Los puntos de Git y calidad de código solo se obtendrán si la práctica está aprobada sin ellos.
Primero se evalúa funcionalidad + arquitectura (mínimo 5 puntos).

1. Funcionalidad (hasta 5 puntos)

- Assets: hasta 1 punto
- Transaccions: hasta 1 punto
- Relación entre colecciones: hasta 1 punto
- Portfolio: hasta 1 punto
- Bonus: hasta 1 punto

2. Arquitectura (hasta 3 puntos)

- Separación de código con una arquitectura como la vista en la teoría.

3. Uso de Git (hasta 1 punto)

- Commits con nombres significativos.
- Historial ordenado, sin "mega commit" de todo al final.

- Uso sensato de ramas si se utilizan.

Solo se suma si la práctica tiene ≥ 5 puntos sin contar Git ni calidad de código.

4. Calidad de código (hasta 1 punto)

- Nombres de variables, funciones y componentes con sentido.
- Código poco duplicado (DRY).

Solo se suma si hay ≥ 5 puntos sin Git ni calidad de código.